18. 실행 레이어 통신 구현 분석

개요

이 문서는 Lighthouse의 실행 레이어(Execution Layer) 통신 구현을 상세히 분석합니다. 실행 레이어는 Ethereum 2.0의 합의 레이어(Consensus Layer)와 실행 레이어(Execution Layer) 간의 통신을 담당하는 핵심 컴포넌트입니다. Engine API를 통해 실행 클라이언트(Geth, Nethermind, Erigon 등)와 통신하며, 블록 빌딩, 페이로드 검증, 포크 선택 업데이트 등의 중요한 기능을 수행합니다.

핵심 아키텍처

ExecutionLayer 구조체

```
// beacon_node/execution_layer/src/lib.rs

#[derive(Clone)]

pub  struct ExecutionLayer<E: EthSpec> {

inner: Arc<Inner<E>>,

}


struct Inner<E: EthSpec> {

engine: Arc<Engine>, // 실행 엔진 인스턴스

builder: ArcSwapOption<BuilderHttpClient>, // MEV 빌더 클라이언트

execution_engine_forkchoice_lock: Mutex<()>, // 포크 선택 동기화 락

suggested_fee_recipient: Option<Address>, // 기본 수수료 수신자
```

```
proposer_preparation_data: Mutex<HashMap<u64, ProposerPreparationDataEntry>>, //
execution_blocks: Mutex<LruCache<ExecutionBlockHash, ExecutionBlock>>, // 실행 블
proposers: RwLock<HashMap<ProposerKey, Proposer>>, // 제안자 정보
executor: TaskExecutor, // 비동기 작업 실행자
payload_cache: PayloadCache<E>, // 페이로드 캐시
last_new_payload_errored: RwLock<bool>, // 마지막 페이로드 에러 상태
}
```

Engine 구조체

```
// beacon_node/execution_layer/src/engines.rs
pub struct Engine {
pub api: HttpJsonRpc, // HTTP JSON-RPC API 클라이언트
payload_id_cache: Mutex<LruCache<PayloadIdCacheKey, PayloadId>>, // 페이로드 ID 키
state: RwLock<State>, // 엔진 상태
latest_forkchoice_state: RwLock<Option<ForkchoiceState>>, // 최신 포크 선택 상태
executor: TaskExecutor, // 작업 실행자
}
```

Engine API 통신 구현

1. HTTP JSON-RPC 클라이언트

```rust
// beacon_node/execution_layer/src/engine_api/http.rs

pub struct HttpJsonRpc {

client: Client, // HTTP 클라이언트

url: SensitiveUrl, // 실행 클라이언트 URL

auth: Option<Auth>, // JWT 인증 정보

id: AtomicU32, // 요청 ID 카운터

}
```

주요 Engine API 메서드들

```rust
// Engine API 상수 정의

pub const ENGINE_NEW_PAYLOAD_V1: &str = "engine_newPayloadV1";

pub const ENGINE_NEW_PAYLOAD_V2: &str = "engine_newPayloadV2";

pub const ENGINE_NEW_PAYLOAD_V3: &str = "engine_newPayloadV3";

pub const ENGINE_NEW_PAYLOAD_V4: &str = "engine_newPayloadV4";

pub const ENGINE_NEW_PAYLOAD_V5: &str = "engine_newPayloadV5";


pub const ENGINE_GET_PAYLOAD_V1: &str = "engine_getPayloadV1";

pub const ENGINE_GET_PAYLOAD_V2: &str = "engine_getPayloadV2";

pub const ENGINE_GET_PAYLOAD_V3: &str = "engine_getPayloadV3";

pub const ENGINE_GET_PAYLOAD_V4: &str = "engine_getPayloadV4";

pub const ENGINE_GET_PAYLOAD_V5: &str = "engine_getPayloadV5";
```

```
pub const ENGINE_FORKCHOICE_UPDATED_V1: &str = "engine_forkchoiceUpdatedV1";

pub const ENGINE_FORKCHOICE_UPDATED_V2: &str = "engine_forkchoiceUpdatedV2";

pub const ENGINE_FORKCHOICE_UPDATED_V3: &str = "engine_forkchoiceUpdatedV3";
```

2. JWT 인증 구현

```
// beacon_node/execution_layer/src/engine_api/auth.rs

pub struct Auth {

jwt_key: JwtKey, // JWT 서명 키

id: Option<String>, // 클라이언트 ID

version: Option<String>, // 클라이언트 버전

}


impl Auth {

pub fn new(jwt_key: JwtKey, id: Option<String>, version: Option<String>) -> Se

Self { jwt_key, id, version }

}

// JWT 토큰 생성

pub fn generate_token(&self) -> Result<String, Error> {

let now = SystemTime::now()

.duration_since(UNIX_EPOCH)

.map_err(|_| Error::InvalidTimestamp)?

.as_secs();
```

```
let claims = Claims {
iat: now,
exp: now + 60, // 1분 유효
id: self.id.clone(),
version: self.version.clone(),
};
self.jwt_key.encode(&claims)
}
}
```

핵심 통신 프로세스

1. 새 페이로드 알림 (New Payload)

```
// beacon_node/execution_layer/src/lib.rs
impl<E: EthSpec> ExecutionLayer<E> {
/// 새로운 실행 페이로드를 실행 클라이언트에 알림
/// engine_newPayload JSON-RPC 호출에 매핑됨
pub async fn notify_new_payload(
&self,
new_payload_request: NewPayloadRequest<'_, E>,
parent_beacon_block_root: Hash256,
) -> Result<PayloadStatus, Error> {
```

```
let  _timer = metrics::start_timer_vec(
&metrics::EXECUTION_LAYER_REQUEST_TIMES,
&[metrics::NEW_PAYLOAD],
);


// 실행 엔진에 새 페이로드 전송
let  result = self
.engine()
.request(|engine| async  move {
engine
.api
.new_payload(new_payload_request, parent_beacon_block_root)
.await
})
.await;


// 에러 상태 업데이트
match &result {
Ok(_) => {
*self.inner.last_new_payload_errored.write().await = false;
}
Err(_) => {
*self.inner.last_new_payload_errored.write().await = true;
```

```
}
}

let payload_status = result?;
// 메트릭 업데이트
metrics::inc_counter_vec(
&metrics::EXECUTION_LAYER_PAYLOAD_STATUS,
&["new_payload", payload_status.status.into()],
);

Ok(payload_status)
}
}
```

2. 포크 선택 업데이트 (Forkchoice Updated)

```
impl<E: EthSpec> ExecutionLayer<E> {
/// 포크 선택 상태를 실행 클라이언트에 업데이트
/// engine_forkchoiceUpdated JSON-RPC 호출에 매핑됨
pub async fn notify_forkchoice_updated(
&self,
head_block_hash: ExecutionBlockHash,
finalized_block_hash: Option<ExecutionBlockHash>,
```

```
payload_attributes: Option<PayloadAttributes>,

forkchoice_update_params: ForkchoiceUpdateParameters,

) -> Result<ForkchoiceUpdatedResponse, Error> {

let  _timer = metrics::start_timer_vec(

&metrics::EXECUTION_LAYER_REQUEST_TIMES,

&[metrics::FORKCHOICE_UPDATED],

);



// 포크 선택 상태 구성

let  forkchoice_state = ForkchoiceState {

head_block_hash,

safe_block_hash: forkchoice_update_params.justified_hash,

finalized_block_hash: finalized_block_hash.unwrap_or_default(),

};



// 실행 엔진에 포크 선택 업데이트 전송

let  response = self

.engine()

.request(|engine| async  move {

engine

.notify_forkchoice_updated(forkchoice_state, payload_attributes)

.await

})
```

```
.await?;


// 메트릭 업데이트
metrics::inc_counter_vec(
&metrics::EXECUTION_LAYER_PAYLOAD_STATUS,
&["forkchoice_updated", response.payload_status.status.into()],
);


Ok(response)
}
}
```

3. 페이로드 가져오기 (Get Payload)

```
impl<E: EthSpec> ExecutionLayer<E> {
/// 실행 클라이언트로부터 페이로드를 가져옴
/// 블록 제안 시 사용됨
pub  async  fn  get_payload(
&self,
payload_parameters: PayloadParameters<'_>,
builder_params: BuilderParams,
) -> Result<ProvenancedPayload<BlockProposalContentsType<E>>, Error> {
let  _timer = metrics::start_timer_vec(
```

```
&metrics::EXECUTION_LAYER_REQUEST_TIMES,
&[metrics::GET_PAYLOAD],
);

// 로컬 실행 엔진에서 페이로드 가져오기
let local_payload_future = self.get_local_payload(payload_parameters);
// 빌더에서 페이로드 가져오기 (MEV-boost)
let builder_payload_future = self.get_builder_payload(builder_params);

// 두 소스 중 먼저 응답하는 것 사용
match tokio::try_join!(local_payload_future, builder_payload_future) {
Ok((local_payload, builder_payload)) => {
// 빌더 페이로드가 더 가치 있는지 확인
if self.should_use_builder_payload(&local_payload, &builder_payload) {
Ok(ProvenancedPayload::Builder(builder_payload))
} else {
Ok(ProvenancedPayload::Local(local_payload))
}
}
Err(e) => Err(e),
}
}
```

```
/// 로컬 실행 엔진에서 페이로드 가져오기
async fn get_local_payload(
&self,
payload_parameters: PayloadParameters<'_>,
) -> Result<BlockProposalContentsType<E>, Error> {
// 페이로드 ID로 페이로드 요청
let payload_id = self.get_payload_id(payload_parameters).await?;
let response = self
.engine()
.request(|engine| async move {
engine.api.get_payload(payload_id).await
})
.await?;


// 페이로드 캐시에 저장
if let Some(cached_payload) = self.cache_payload((
response.execution_payload_ref(),
response.blobs_bundle.as_ref(),
)) {
metrics::inc_counter(&metrics::EXECUTION_LAYER_PAYLOAD_CACHED);
}


Ok(BlockProposalContentsType::Full(response.try_into()?))
```

```
}

}
```

블록 빌딩 프로세스

1. 제안자 준비 (Proposer Preparation)

```
impl<E: EthSpec> ExecutionLayer<E> {

/// 검증자의 제안자 준비 데이터 업데이트

pub  async  fn  update_proposer_preparation<'a, I>(

&self,

update_epoch: Epoch,

proposer_data: I,

) where

I: IntoIterator<Item = (&'a ProposerPreparationData, &'a Option<u64>)>,

{

let  mut  proposer_preparation_data = self.proposer_preparation_data().await;



for (preparation_entry, gas_limit) in  proposer_data {

let  new = ProposerPreparationDataEntry {

update_epoch,

preparation_data: preparation_entry.clone(),

gas_limit: *gas_limit,
```

```
};

match  proposer_preparation_data.entry(preparation_entry.validator_index) {
Entry::Occupied(mut  entry) => {
if  entry.get_mut().update(new) {
metrics::inc_counter(&metrics::EXECUTION_LAYER_PROPOSER_DATA_UPDATED);
}
}
Entry::Vacant(entry) => {
entry.insert(new);
metrics::inc_counter(&metrics::EXECUTION_LAYER_PROPOSER_DATA_UPDATED);
}
}
}
}
}
```

2. 페이로드 속성 구성

```
// beacon_node/execution_layer/src/engine_api.rs
#[superstruct(
variants(V1, V2, V3),
variant_attributes(derive(Clone, Debug, Eq, Hash, PartialEq),),
```

```
)]
#[derive(Clone, Debug, Eq, Hash, PartialEq)]
pub struct PayloadAttributes {
#[superstruct(getter(copy))]
pub timestamp: u64, // 블록 타임스탬프
#[superstruct(getter(copy))]
pub prev_randao: Hash256, // 이전 랜덤 값
#[superstruct(getter(copy))]
pub suggested_fee_recipient: Address, // 제안된 수수료 수신자
#[superstruct(only(V2, V3))]
pub withdrawals: Vec<Withdrawal>, // 출금 목록 (Capella+)
#[superstruct(only(V3), partial_getter(copy))]
pub parent_beacon_block_root: Hash256, // 부모 비콘 블록 루트 (Deneb+)
}

impl PayloadAttributes {
pub fn new(
timestamp: u64,
prev_randao: Hash256,
suggested_fee_recipient: Address,
withdrawals: Option<Vec<Withdrawal>>,
parent_beacon_block_root: Option<Hash256>,
) -> Self {
```

```
match  withdrawals {
Some(withdrawals) => match  parent_beacon_block_root {
Some(parent_beacon_block_root) => PayloadAttributes::V3(PayloadAttributesV3 {
timestamp,
prev_randao,
suggested_fee_recipient,
withdrawals,
parent_beacon_block_root,
}),
None => PayloadAttributes::V2(PayloadAttributesV2 {
timestamp,
prev_randao,
suggested_fee_recipient,
withdrawals,
}),
},
None => PayloadAttributes::V1(PayloadAttributesV1 {
timestamp,
prev_randao,
suggested_fee_recipient,
}),
}
}
```

```
}
```

페이로드 캐시 시스템

PayloadCache 구현

```
// beacon_node/execution_layer/src/payload_cache.rs

pub struct PayloadCache<E: EthSpec> {

payloads: Mutex<LruCache<PayloadCacheId, FullPayloadContents<E>>>,

}


impl<E: EthSpec> PayloadCache<E> {

/// 페이로드를 캐시에 저장

pub fn put(&self, payload: FullPayloadContents<E>) -> Option<FullPayloadConten

let root = payload.payload_ref().tree_hash_root();

self.payloads.lock().put(PayloadCacheId(root), payload)

}


/// 루트로 페이로드 검색

pub fn get(&self, hash: &Hash256) -> Option<FullPayloadContents<E>> {

self.payloads.lock().get(&PayloadCacheId(*hash)).cloned()

}
```

```
/// 페이로드를 캐시에서 제거하고 반환
pub fn pop(&self, root: &Hash256) -> Option<FullPayloadContents<E>> {
self.payloads.lock().pop(&PayloadCacheId(*root))
}
}
```

에러 처리 및 복구

1. 엔진 상태 관리

```
// beacon_node/execution_layer/src/engines.rs
#[derive(Copy, Clone, PartialEq, Debug, Eq, Default)]
enum EngineStateInternal {
#[default]
Synced, // 동기화됨
Offline, // 오프라인
Syncing, // 동기화 중
AuthFailed, // 인증 실패
}

#[derive(Debug, Clone, PartialEq, Eq, Copy)]
pub enum EngineState {
Online, // 온라인
```

```
Offline, // 오프라인
}

impl From<EngineStateInternal> for EngineState {
fn  from(state: EngineStateInternal) -> Self {
match  state {
EngineStateInternal::Synced | EngineStateInternal::Syncing => EngineState::Onlin
EngineStateInternal::Offline | EngineStateInternal::AuthFailed => EngineState::O
}
}
}
```

2. 상태 감시 및 알림

```
impl Engine {
/// 엔진 상태 감시 채널 제공
pub  async  fn  watch_state(&self) -> WatchStream<EngineState> {
self.state.read().await.watch()
}

/// 엔진 상태 업데이트
async  fn  update_state(&self, new_state: EngineStateInternal) {
self.state.write().await.update(new_state);
```

```
}

/// 엔진 상태 확인 (헬스체크)
pub  async  fn  upcheck(&self) {
match  self.api.upcheck().await {
Ok(_) => {
self.update_state(EngineStateInternal::Synced).await;
}
Err(EngineApiError::Auth(_)) => {
self.update_state(EngineStateInternal::AuthFailed).await;
}
Err(_) => {
self.update_state(EngineStateInternal::Offline).await;
}
}
}
}
}
```

3. 감시 루틴 (Watchdog)

```
impl<E: EthSpec> ExecutionLayer<E> {
/// 실행 엔진을 온라인 상태로 유지하는 감시 루틴 시작
pub  fn  spawn_watchdog_routine<S: SlotClock + 'static>(&self, slot_clock: S) {
```

```
let watchdog = |el: ExecutionLayer<E>| async move {
// 즉시 한 번 실행
el.watchdog_task().await;

// 주기적 업데이트 루프 시작
loop {
el.spawn(
|el| async move { el.watchdog_task().await },
"exec_watchdog_task",
);
sleep(slot_clock.slot_duration()).await;
}
};

self.spawn(watchdog, "exec_watchdog");
}

/// 감시 작업 단일 실행
pub async fn watchdog_task(&self) {
self.engine().upcheck().await;
}
}
```

MEV-Boost 통합

1. 빌더 클라이언트 설정

```
impl<E: EthSpec> ExecutionLayer<E> {
/// 초기화 후 빌더 URL 설정
pub fn set_builder_url(
&self,
builder_url: SensitiveUrl,
builder_user_agent: Option<String>,
builder_header_timeout: Option<Duration>,
disable_ssz: bool,
) -> Result<(), Error> {
let builder_client = BuilderHttpClient::new(
builder_url.clone(),
builder_user_agent,
builder_header_timeout,
disable_ssz,
)
.map_err(Error::Builder)?;
info!(
?builder_url,
local_user_agent = builder_client.get_user_agent(),
```

```
ssz_disabled = disable_ssz,
"Using external block builder"
);
self.inner.builder.swap(Some(Arc::new(builder_client)));
Ok(())
}

/// 빌더 클라이언트 접근
pub fn builder(&self) -> Option<Arc<BuilderHttpClient>> {
self.inner.builder.load_full()
}
}
```

2. 빌더 vs 로컬 페이로드 선택

```
impl<E: EthSpec> ExecutionLayer<E> {
/// 빌더 페이로드를 사용할지 결정
fn should_use_builder_payload(
&self,
local_payload: &BlockProposalContentsType<E>,
builder_payload: &BlockProposalContentsType<E>,
) -> bool {
// 블록 가치 비교
```

```
let local_value = local_payload.block_value();

let builder_value = builder_payload.block_value();

// 빌더 페이로드가 더 가치 있는 경우 사용

builder_value > local_value

}

}
```

성능 최적화

1. 연결 풀링 및 재사용

```
// HTTP 클라이언트 설정

impl HttpJsonRpc {

pub fn new_with_auth(

url: SensitiveUrl,

auth: Auth,

execution_timeout_multiplier: Option<u32>,

) -> Result<Self, Error> {

let client = Client::builder()

.timeout(Duration::from_secs(EXECUTION_ENGINE_REQUEST_TIMEOUT))

.pool_idle_timeout(Duration::from_secs(30))

.pool_max_idle_per_host(8)

.build()
```

```
.map_err(Error::HttpClient)?;


Ok(Self {
client,
url,
auth: Some(auth),
id: AtomicU32::new(STATIC_ID),
})
}
}
```

2. 비동기 요청 처리

```
impl Engine {
/// 비동기 요청 처리 with 타임아웃
pub  async  fn  request<F, G, R>(&self, request_fn: F) -> Result<R, EngineError>
where
F: FnOnce(&HttpJsonRpc) -> G,
G: Future<Output = Result<R, EngineApiError>>,
{
let  result = tokio::time::timeout(
Duration::from_secs(REQUEST_TIMEOUT_SECONDS),
request_fn(&self.api),
```

```
)
.await;

match  result {
Ok(Ok(response)) => {
self.update_state(EngineStateInternal::Synced).await;
Ok(response)
}
Ok(Err(api_error)) => {
match &api_error {
EngineApiError::Auth(_) => {
self.update_state(EngineStateInternal::AuthFailed).await;
}
_ => {
self.update_state(EngineStateInternal::Offline).await;
}
}
Err(EngineError::Api { error: api_error })
}
Err(_timeout) => {
self.update_state(EngineStateInternal::Offline).await;
Err(EngineError::Offline)
}
```

```
}

}

}
```

메트릭 및 모니터링

1. 성능 메트릭

```
// beacon_node/execution_layer/src/metrics.rs

pub  static EXECUTION_LAYER_REQUEST_TIMES: LazyLock<HistogramVec> = LazyLock::ne

try_create_histogram_vec(

"execution_layer_request_times",

"Duration of execution layer requests",

&["method"],

)

.unwrap()

});


pub  static EXECUTION_LAYER_PAYLOAD_STATUS: LazyLock<IntCounterVec> = LazyLock::

try_create_int_counter_vec(

"execution_layer_payload_status",

"Count of payload status responses",

&["method", "status"],
```

```
)
.unwrap()
});

// 메트릭 상수
pub const NEW_PAYLOAD: &str = "new_payload";
pub const FORKCHOICE_UPDATED: &str = "forkchoice_updated";
pub const GET_PAYLOAD: &str = "get_payload";
```

2. 상태 모니터링

```
impl<E: EthSpec> ExecutionLayer<E> {
/// 실행 레이어가 오프라인이거나 에러 상태인지 확인
pub async fn is_offline_or_erroring(&self) -> bool {
self.engine().is_offline().await || *self.inner.last_new_payload_errored.read().
}

/// 동기화 상태 확인 (알림용)
pub async fn is_synced_for_notifier(&self, current_slot: Slot) -> bool {
let synced = self.is_synced().await;
if synced {
if let Ok(Some(block)) = self
.engine()
```

```
.api
.get_block_by_number(BlockByNumberQuery::Tag(LATEST_TAG))
.await
{
if  block.block_number == 0 && current_slot > 0 {
return false;
}
}
}
synced
}
}
```

설정 및 초기화

1. 설정 구조체

```
#[derive(Debug, Default, Clone, Serialize, Deserialize)]
pub  struct Config {
/// 실행 레이어 노드 엔드포인트 URL
pub  execution_endpoint: Option<SensitiveUrl>,
/// 빌더 API 서비스 URL
pub  builder_url: Option<SensitiveUrl>,
```

```rust
/// 빌더 API 헤더 요청 타임아웃
pub builder_header_timeout: Option<Duration>,
/// 빌더 API 사용자 에이전트
pub builder_user_agent: Option<String>,
/// 빌더에서 SSZ 요청 비활성화
pub disable_builder_ssz_requests: bool,
/// JWT 시크릿 파일 경로
pub secret_file: Option<PathBuf>,
/// 기본 수수료 수신자
pub suggested_fee_recipient: Option<Address>,
/// JWT ID
pub jwt_id: Option<String>,
/// JWT 버전
pub jwt_version: Option<String>,
/// 기본 데이터 디렉토리
pub default_datadir: PathBuf,
/// 실행 타임아웃 배수
pub execution_timeout_multiplier: Option<u32>,
}
```

2. 초기화 프로세스

```rust
impl<E: EthSpec> ExecutionLayer<E> {
```

```
/// 설정으로부터 ExecutionLayer 인스턴스 생성
pub fn from_config(config: Config, executor: TaskExecutor) -> Result<Self, Err
let execution_url = config.execution_endpoint.ok_or(Error::NoEngine)?;
// JWT 키 로드 또는 생성
let secret_file = config.secret_file
.unwrap_or_else(|| config.default_datadir.join(DEFAULT_JWT_FILE));
let jwt_key = if secret_file.exists() {
// 기존 파일에서 시크릿 읽기
std::fs::read_to_string(&secret_file)
.and_then(|s| {
let secret = JwtKey::from_slice(
&hex::decode(strip_prefix(s.trim_end()))?
)?;
Ok(secret)
})
.map_err(Error::InvalidJWTSecret)
} else {
// 새 파일 생성 및 랜덤 시크릿 작성
warn!(path = %secret_file.display(), "No JWT found on disk. Generating");
std::fs::File::options()
.write(true)
.create_new(true)
.open(&secret_file)
```

```
.and_then(|mut  f| {
let  secret = JwtKey::random();
f.write_all(secret.hex_string().as_bytes())?;
Ok(secret)
})
.map_err(Error::InvalidJWTSecret)
}?;


// 엔진 생성
let  engine = {
let  auth = Auth::new(jwt_key, config.jwt_id, config.jwt_version);
let  api = HttpJsonRpc::new_with_auth(
execution_url,
auth,
config.execution_timeout_multiplier,
)?;
Engine::new(api, executor.clone())
};


// Inner 구조체 초기화
let  inner = Inner {
engine: Arc::new(engine),
builder: ArcSwapOption::empty(),
```

```
execution_engine_forkchoice_lock: Mutex::new(()),
suggested_fee_recipient: config.suggested_fee_recipient,
proposer_preparation_data: Mutex::new(HashMap::new()),
proposers: RwLock::new(HashMap::new()),
execution_blocks: Mutex::new(LruCache::new(EXECUTION_BLOCKS_LRU_CACHE_SIZE)),
executor,
payload_cache: PayloadCache::default(),
last_new_payload_errored: RwLock::new(false),
};

let  el = Self {
inner: Arc::new(inner),
};

// 빌더 URL 설정 (있는 경우)
if  let Some(builder_url) = config.builder_url {
el.set_builder_url(
builder_url,
config.builder_user_agent,
config.builder_header_timeout,
config.disable_builder_ssz_requests,
)?;
}
```

```
Ok(el)

}

}
```

결론

Lighthouse의 실행 레이어 통신 구현은 Engine API를 통해 실행 클라이언트와의 안정적이고 효율적인 통신을 제공합니다. 주요 특징은 다음과 같습니다:

1. **강력한 에러 처리**: 연결 실패, 인증 오류, 타임아웃 등 다양한 에러 상황에 대한 복구 메커니즘
2. **성능 최적화**: 연결 풀링, 캐싱, 비동기 처리를 통한 높은 성능
3. **MEV-Boost 통합**: 빌더 API와의 통합으로 MEV 기회 활용
4. **포괄적인 모니터링**: 상세한 메트릭과 상태 추적
5. **유연한 설정**: 다양한 실행 클라이언트와 네트워크 환경 지원

이러한 구현을 통해 Lighthouse는 Ethereum 2.0의 합의 레이어와 실행 레이어 간의 원활한 통신을 보장하며, 안정적인 블록 생산과 검증을 수행할 수 있습니다.